

Distributed Pair Programming: A Systematic Literature Review



Bernardo José da Silva Estácio*, Rafael Prikladnicki

Pontifícia Universidade Católica do Rio Grande do Sul – PUCRS, Faculdade de Informática (Facin), 6681 Ipiranga Avenue, Building 32, 90619-900 Porto Alegre, Brazil

ARTICLE INFO

Article history:

Received 9 October 2014
Received in revised form 30 January 2015
Accepted 21 February 2015
Available online 6 March 2015

Keywords:

Extreme Programming
Distributed Software Development
Pair Programming
Distributed Pair Programming

ABSTRACT

Context: Geographically distributed teams have adopted agile practices as a work strategy. One of these practices is Distributed Pair Programming (DPP). DPP consists in two developers working remotely on the same design, algorithm or code.

Objective: In this paper we sought to identify and synthesize papers that describe and analyze DPP both from teaching and practice perspectives.

Method: We conducted a Systematic Literature Review to search for empirical evidence in eight digital libraries.

Results: Most of the 34 DPP primary studies identified explore DPP from a teaching perspective. We found that DPP requires a specific infrastructure, but the existing studies do not explore the impact of the distribution in the details. There are many tools proposed that support DPP practice, but few of them are evaluated within a software development team.

Conclusion: We need more studies that explore the effects of Pair Programming in the context of Distributed Software Development, such as coordination and communication. Most of the studies do not empirically evaluate DPP in industry. There is also a need to propose guidelines to use DPP in industry and as a teaching strategy.

© 2015 Elsevier B.V. All rights reserved.

Contents

1. Introduction	2
2. Distributed Pair Programming	2
3. Related work	2
4. Methodology	3
4.1. Literature search	3
4.2. Selection of the studies	3
4.3. Quality assessment of the studies	3
4.4. Data extraction and synthesis	4
5. Results	4
5.1. Quantitative analysis	4
5.2. Qualitative analysis	5
5.2.1. RQ. What is known about the use of Distributed Pair Programming?	5
5.2.2. RQ2. Under what conditions Distributed Pair Programming works?	6
5.2.3. RQ3. Which are the main tools that can support the practice of DPP?	7
6. Discussion and conclusions	7
6.1. Conclusions based on quantitative analysis	7
6.2. Conclusions based on qualitative analysis	8
6.3. Limitations	8

* Corresponding author.

E-mail addresses: bernardoestacio@acad.pucrs.br (B.J. da Silva Estácio), rafaelp@pucrs.br (R. Prikladnicki).

Acknowledgments	8
Appendix A. Papers extracted from the SLR	8
References	9

1. Introduction

Pair Programming (PP) is one of the primary practices of Extreme Programming (XP) [2]. As the name suggests, it is a practice that involves two programmers at one computer, working collaboratively [19]. In a PP session, a programmer acts as the driver and implements the code controlling the keyboard and mouse. Another programmer acts as the observer or navigator and is responsible for reviewing the code, preventing and identifying logical and syntactical errors in the code at the same time.

A strong collaboration and a good communication are requirements for a successful execution of Pair Programming. In a PP session, both programmers should be in a continuous conversation to discuss possible solutions and code errors [19]. The successful of PP has also a direct relationship with the workplace, due arrangement of the pairs [30]. The feasibility of using the practice of Pair Programming is also related to the complexity of the task [11].

On the other hand, the literature reports several benefits of the practice of Pair Programming: productivity product quality due to the high number of defects found in the ongoing review of the pairs, collaboration and communication within the team and the improvement of confidence and motivation between the programmers [11].

Many studies involving the practice of Pair Programming have been developed, focusing both the practice of PP or PP as a tool for teaching programming. In the context of education, research results showed that PP creates an environment that leads the students to a better learning and social interaction, increasing the grades, confidence and the interest in programming subjects [25]. As for the practice of PP, results reported an improvement in code quality and performance at work, though some critics have also been reported on effort, costs of staff and personality clashes between the programmers [11].

In parallel, in recent years the IT globalization has lead the companies to distribute their software development around the world, emerging the concept of Distributed Software Development (DSD), or Global Software Engineering (GSE) [16]. In this context, Pair Programming also started to be adopted by distributed teams [20], emerging the concept of Distributed Pair Programming (DPP) [1]. In DPP, two programmers collaborate to develop software using tools that allows screen sharing and communication such as audio, text and video.

The goal of this work is to identify and analyze the state of the art of DPP, both from the teaching and practice perspective. To reach this goal we have conducted a systematic review of the literature. This paper is organized as follows: in Section 2 we present the background of the work. In Section 3, we detail the related works, while in Section 4 we present our methodology. In Section 5 we set out the results, while in Section 6 we discuss the findings and future opportunities in this area.

2. Distributed Pair Programming

The combination of practicing PP with geographically distributed team members has created the Distributed Pair Programming. Some studies have investigated the effects of the DPP. Baheti reported that the benefits of PP are the same DPP, such as productivity and code quality [1]. Another benefit of DPP is that due to its

characteristics, it helps to promote the work and communication within distributed teams [1].

However, the focus on aspects such as infrastructure should be redoubled, and this is reflected in the adoption of specific tools for practice. Ho said that the use of specific tools for DPP helps makes the practice faster [17]. Canfora reports that a specific tool prevents the programmer of switching between different tools, spending time [5]. DPP tools have to provide a good communication channel through chat and screen sharing between the developers [5]. Canfora also reported that code quality suffers a small decrease due to infrastructure issues, but they have found no evidence to report an increase in the effort of the team [5].

In industry, Rosen [23] conducted a case study of a pilot project in two branches of a German company. They reported initial results, indicating that the practice could present positive effects in communication (a more integration of the developers using DPP sessions, discussing developing issues and suggesting solutions), knowledge transfer (support the feedback and teaching of novice developers) and code quality (less defects. As negative effects perceived in the case study, the authors cited distraction, lack of fulfillment of the role of DPP (either driver or observer) and conflict of goals among the developers during the session of DPP.

DPP also presented benefits in teaching programming. Some positive effects were found in relation to learning, code quality [34] and the programmer performance [13]. In an experiment conducted by Hanks [13] students who used the DPP had performed as well as students who were co-located with pairs, passing the course with similar grades.

3. Related work

Systematic Literature Review (SLR) is part of Evidence-Based Software Engineering and aim to identify, combine and assess the evidence from the primary studies, following an explicit, rigorous and systematic method [18]. We did not find in the literature any SLR covering DPP. However, we did find studies about PP. Table 1 presents a comparative analysis of the SLR presented in this paper and the other three PP systematic studies.

The studies we found were published between 2009 and 2011. In terms of the period of the primary studies found, a range of nine years by Dyba et al. study [10], 1998–2007, and a range of eight years specified by Salleh et al. [25], 1999–2007. Brereton et al. [3] et al. did not report the period of the primary studies.

The quality assessment represents an important aspect of a systematic study to evaluate the evidence presented [18], only Salleh et al. [25] conducted a quality assessment. The number of primary studies of each paper and the number of data sources used diverged. The common data sources used for all the studies were: ACM Digital Library, IEEE Xplore and ISI Web of Science.

In relation to the context, Dyba et al. [11] analyzed the effects of Pair versus Solo Programming that have been reported in the literature. The two SLR covered the Teaching aspects in PP. Salleh et al. presented evidences of PP as a pedagogical tool in higher education CE/SE courses. In the same way, Brereton et al. brought some results about the effectiveness of PP in introductory courses, presenting evidence in students' confidence, motivation and retention.

Table 1
Related work.

Study	Type	Year	Quality assessment of primary studies	#number of primary studies/ number of data sources	Period	Context
Dyba et al.	Meta-analysis	2009	No quality assessment	18/4	1998–2007	Effectiveness of PP
Brereton et al.	SLR	2009	No quality assessment	28/4	–	Teaching in PP
Salleh et al.	SLR	2011	Based on evaluation criteria of six studies	73/12	1999–2007	Teaching in PP
Estácio and Prikladnicki (this study)	SLR	2014	Based on evaluation criteria proposed by Dyba and Salleh	34/8	2001–2013	Effects and conditions for adopting DPP

Our SLR is not focused only in the teaching perspective, but we also explored DPP in the industry context. We also wanted to identify the settings needed to adopt the practice and the tools proposed and published in the literature.

4. Methodology

In this study, the SLR followed the guidelines proposed by Kitchenham. Two researchers participated in the review process [18]. A graduate student was the primary reviewer, which was responsible for developing the research protocol, search strategy, selection of studies, quality assessment of studies, extraction and synthesis of data and reporting of results. The other reviewer was responsible for validating the review protocol, monitoring the results of the search and selection processes and verifying the data of the quality assessment and the data of information extraction. Fig. 1 presents the methodology followed in SLR.

The research questions that guided the Systematic Literature Review were:

- RQ1: What is known about the use of Distributed Pair Programming?
- RQ2: Under what conditions Distributed Pair Programming works?
- RQ3: Which are the main tools to support Distributed Pair Programming?

The first research question (RQ1) investigates the main variables and effects (benefits and disadvantages) of DPP, aiming to identifying the contributions of the existing DPP studies. From these studies, we synthesize and analyze the conditions where DPP was (un) successfully adopted (RQ2). As the use of a tool is essential in DPP, in the last question (RQ3) we investigate the existing DPP tools and the requirements needed for practicing DPP.

4.1. Literature search

We conducted manual searches with the string (“Virtual Pair Programming” or “Remote Pair Programming” or “Distributed

Pair Programming”) in eight digital libraries, as following: ACM Digital Library; DBLP; IEEE Xplore; Google Scholar; Compedex; Scopus; ISI Web of Knowledge and Willey. Similar to the study conducted by Salleh [25] the main digital library selected was Scopus. Therefore, each paper returned from the other libraries was compared with the preliminary list of the papers returned from Scopus in order to avoid duplication.

4.2. Selection of the studies

We defined as inclusion criteria only studies that used DPP in the context of the software development. The search covered only studies that were published between 2001 and January 2014. We defined 2001 as initial date due to the publication of the Agile Manifesto, which was the basis for agile methods such as XP. The end date January 2014 is related to the period of the final selection of the papers. The SLR was focused on studies in English only.

As exclusion criteria, we adopted the following:

- Studies in other languages than English.
- Items that do not address the practice of Distributed Pair Programming.
- Tutorials and lectures.

The selection process consisted of two phases. In the first phase, for each paper we reviewed the title, abstract and keywords. Relevant papers were selected and in the second phase each relevant study was read in full to determine if the paper was a primary study for this SLR.

4.3. Quality assessment of the studies

To support data extraction one quality checklist was developed. The questions and the score were adapted from a SLR conducted by Dyba [10]. The checklist (Table 2) consists of seven questions to be answered following this scale: Yes = 1 point, partially = 0.5 and No = 0 points. The total result of each study has a range from 0 (very bad) to 7 (very good).

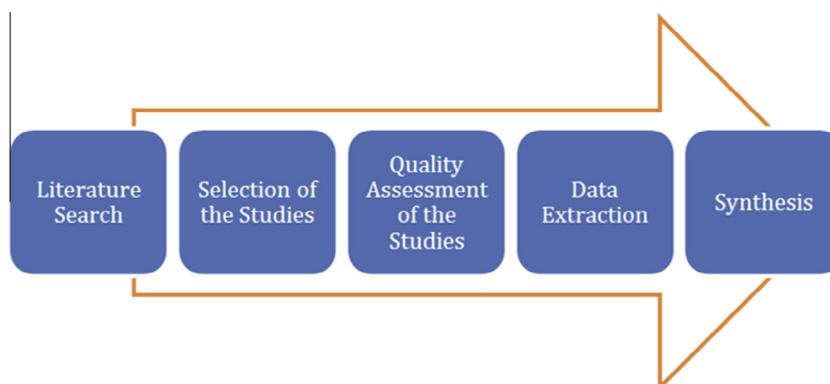


Fig. 1. Methodology followed.

Table 2
Checklist for Quality Assessment of the Studies.

Item	Answer
1 – The paper is properly referenced (presents work related and literature review)?	() Yes () Partly () No
2 – The objective of the research is clear ^(a) ?	() Yes () Partly () No
3 – The research method was appropriate for achieving the objectives of the research?	() Yes () Partly () No
4 – There is a clear description of the context in which the research was conducted?	() Yes () Partly () No
5 – Data collection was done properly ^(b) ?	() Yes () Partly () No
6 – The data analysis was performed properly ^(c) ?	() Yes () Partly () No
7 – The results have credibility ^(d) ?	() Yes () Partly () No

Indicators/Quality Guidelines.

^a The paper is based on a survey or is it just a set a report of lessons learned based on the opinion of an expert?

^b There is a discussion of: Who conducted the data collection? Procedures/documents used to collect/Audio or video recording of interviews/discussions/conversations (if you have not registered for a justification given?). As the methods of field study or case study applied may have influenced the data collected.

^c There is a thorough description of the data analysis? Have sufficient data to support the results? The contradictory data have been considered? Quality control methods were used to check the results?

^d Results are supported by data/empirical studies (i.e., the reader can see how the researcher arrived at his/her conclusions, the methodology of analysis and interpretation are evident).

4.4. Data extraction and synthesis

To extract the data we used a data extraction form developed in MS Excel. The items of the form were: Paper; Year; Author; Source; (where the paper was published); Type (Journal, Conference, Workshop); Purpose; Context (Educational/Industry); Contribution; Tools/Infrastructure; Future Work; Methodology; Status (Included or Not Included) and Justification (referring to status). We used the web tool CiteULike to manage the reading and selection of the papers.

5. Results

The initial search with the string “Remote Pair Programming” OR “Virtual Pair Programming” OR “Distributed Pair Programming” returned 435 papers, but only 147 were considered potentially relevant based on the reading of the title and abstract. All 147 papers were read in full and after the application of inclusion and exclusion criteria and the removal of duplicates, we selected 34 papers for quality assessment.

Table 3 shows the quality score of 34 papers. Most of the papers (16) were assessed as Regular, and 1 study as Very Good.

5.1. Quantitative analysis

Fig. 2 shows the classification of the papers based on the categories proposed by Wieringa et al. [33]. Most of the papers have used controlled experiments, being classified as experience paper (22 papers). On the other hand, 5 papers were classified as a

Classification by Wieringa (2005)

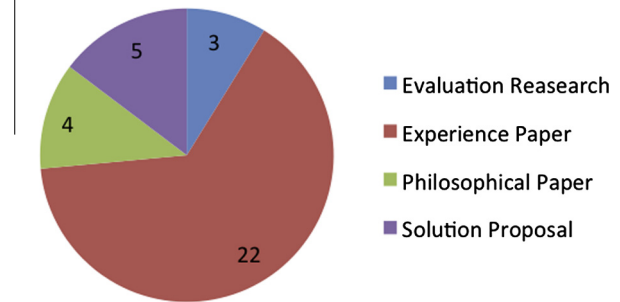


Fig. 2. Classification followed Wieringa et al. [33].

solution proposal, as they propose DPP tools. Only 3 papers explore DPP in a case study, and were classified as Evaluation Research.

Although the first paper related to the DPP was published in 2002, the highest number of publications of the practice occurred between 2009 and 2010. Fig. 3 shows the distribution of the primary studies over the years.

In relation of the publication type, 20 papers were published in conferences, while 7 papers came from journals and other 7 came from workshops (Fig. 4).

As PP has an essential benefit in software engineering education, most of the publications were published in conferences and workshops of this area, such as ITiCSE and the Symposium on Computer Science Education. The XP Conference also presents a high number of DPP papers published. Table 4 summarizes the main events where the papers were published.

We also classified the papers based on their primary focus: practice (when the focus of the paper was to investigate the main effects and variables related to DPP) or education (when the

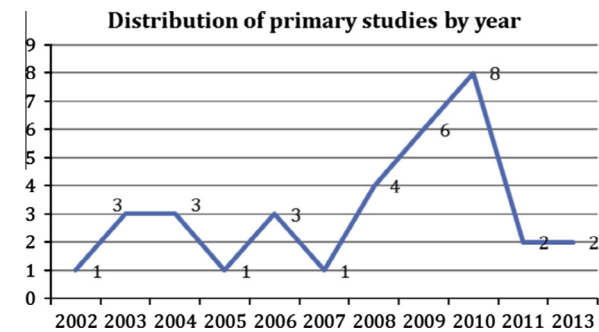


Fig. 3. Distribution of primary studies by publication year.

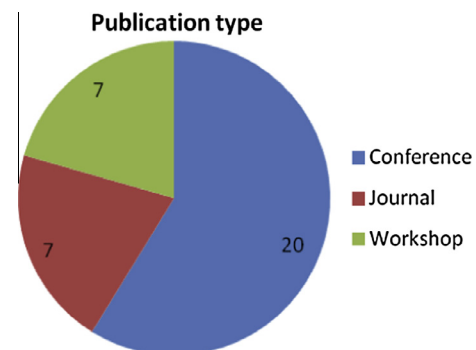


Fig. 4. Distribution of primary studies by publication type.

Table 3
Quality assessment results.

	Very Poor (<2)	Poor (2 - < 3)	Regular (3 - < 5)	Good(5 - <=6)	Very good (>6)	Total
Number of studies	0	12	16	5	1	34

Table 4
Main events publishing studies related to DPP.

Event	Number of papers
Innovation and Technology in Computer Science Education (ITiCSE)	4
International Conference on Agile Software Development (XP)	4
Symposium on Computer Science Education	4
Chase Workshop	2
International Workshop on Education Technology and Computer Science	2

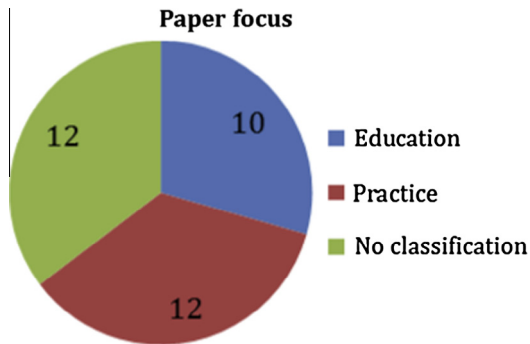


Fig. 5. Focus of DPP paper.

authors aimed to evaluate the use of DPP in the teaching process). When the paper did not present empirical results, then the focus was not clear, i.e. tools, and for this reason we labeled as no classification. Fig. 5 presents the results.

5.2. Qualitative analysis

The qualitative analysis aimed to characterize the selected studies in order to answer the research questions defined in the SLR protocol.

5.2.1. RQ. What is known about the use of Distributed Pair Programming?

5.2.1.1. Practice. In 2002, Baheti assessed the DPP effectiveness through an experiment involving students in both collocated and distributed teams, where some teams have used Pair Programming and others not [1]. Two metrics were used: quality (students' grades) and productivity (LLOC per hour). The results showed that in terms of the two metrics, DPP was statistically equivalent for both teams. The feedback given by the students indicated that DPP helps to promote the work and communication within distributed teams. This experiment was the first to indicate that DPP is feasible and is an efficient method for dealing with Distributed Software Development.

After this experiment, other studies were conducted in order to analyze DPP efficiency. Canfora held two [4]. The first experiment was performed in 11-week and involved students who formed eight pairs collocated and eight distributed. The experiment aimed to analyze the impact of the distribution of PP according to productivity metrics (in terms of lines of code) and code quality (students' grades). The results indicated that the pairs tended to work alone. Four factors were identified to explain this behavior: failure to establish a working protocol, the conflict of ideas among the pairs, problems with the chat software and different levels of experience between the pairs.

In order to collect more information, the second experiment was a replication of the first [5]. In the replication, the pair rejection was lower. The results showed that the quality has declined in DPP, because there were collaboration and infrastructure issues. Canfora reported two factors to the DPP success: appropriate communication and collaboration support.

By the analysis of Collece, a specific tool for DPP, Duque [9] reported that code quality of students who enjoyed DPP was higher than the solo programmers, measuring by the syntactical errors. On the other hand, DPP required more time to complete a task, spending more effort in communication and coordination interactions between the students. However, there is a reduction in the time to edit a source code under construction.

The pilot project conducted by Rosen in two subsidiaries of a German company is the only case study identified that involves DPP with industry professionals [23]. Positive effects were reported such as improvement in communication, knowledge transfer and code quality. As negative effect were cited: distraction, lack of fulfillment of the role (driver or observer) in DPP (either driver or observer) and goals conflict during the DPP session between the pairs.

Fig. 6 shows the main variables found in the literature regarding DPP with the focus on practice. The range in the figure is related to the number of studies that report positive and negative effects per each variable. For example, we found five studies of DPP practice related to code quality. Two of them [4,5] reported negative effects and three of them [1,9,23] reported positive effects. Four studies were conducted with students, and one study was conducted with professionals.

5.2.1.2. Teaching. Hanks conducted a student experiment about checking the performance of a tool to support DPP in an introductory programming course [13]. The results showed that students who used DPP had performed as well as students with pairs who were co-located, being approved in the course with similar grades. Another metric used was the confidence level, which was statistically similar.

The experiment conducted by Zin et al. [35] evaluated DPP effectiveness in class. The main limitation of this study is that DPP was used asynchronously due to the system used at the university. The results showed that DPP helped the student in terms of confidence and learning. Students also indicated that it would be more interesting to use DPP synchronously through specific tools that facilitate the communication between the pairs.

The study conducted by Edwards et al. [12] aimed to evaluate DPP effectiveness in the curriculum of a Computer Science course. At the end of the course, students answered a survey about their performance. PP had a range of 80–90% approval by the students, while DPP had a slightly lower approval (72–80%). The authors reported that there is a difference between PP and DPP, especially in relation to the methodology, but the use of a specific tool is a feasible solution to overcome the differences found.

Zacharis conducted a study with students to investigate the effectiveness of DPP on student performance and motivation in a Java introductory course [34]. In this study DPP comparison was made with the individual programming and the results indicated that students who used DPP have 50% fewer defects than those who did not used DPP, demonstrating the improvement of code quality, and the productivity in the metric LOC/h [34]. The performance also increased with students' grades. Students also completed a survey at the end of the course, reporting that in DPP they had more motivation and satisfaction. Fig. 7 shows the variables about DPP with the focus on teaching. As in Fig. 6, the range in the figure is related to the number of studies that report positive and negative effects in each variable.

DPP practice							
Code Quality		[4]	[5]		[1]	[9]	[23]
Knowledge					[4]	[23]	
Productivity				[1]	[5]		
Communication					[1]	[23]	
Edition of a source code					[9]		
Distraction			[23]				
Fulfillment role			[23]				
Effort			[9]				
Goal conflicts			[23]				
Students		-2	-1	0	1	2	3
Professionals		Negative		No effect	Positive		

Fig. 6. DPP practice found in the literature.

Using DPP for teaching programming						
Performance			[12]		[13]	[34]
Grades					[13]	
Productivity					[34]	
Motivation					[34]	[12]
Confidence					[35]	
Learning					[35]	
Students		-2	-1	0	1	2
		Negative		No effect	Positive	

Fig. 7. Using DPP for teaching programming.

5.2.2. RQ2. Under what conditions Distributed Pair Programming works?

Few studies have presented empirical evidence about DPP. In addition, most of the studies reported about the practice of DPP were conducted with students.

5.2.2.1. DPP and other types of programming. Empirical studies compared DPP with different types of programming. As DPP is a derived from PP, most of the studies aimed to compare the efficiency between both practices such as observed in [1,4,5,12]. Some of the studies have a focus in comparing DPP with solo (individual) programming, as identified in [9,34].

5.2.2.2. Infrastructure. The main requirement for adopting a specific tool for DPP indicates that the use of these DPP tools help to improve the practice and to implement it more quickly [17]. Canfora [5] reported that a specific tool prevents the developer to switch between different tools, and spending a lot of time on it. These tools have to provide a good synchronous communication channel through chat and screen sharing between developers. Rosen [23] reports that the screen sharing and video are important

features in a tool-specific DPP to minimize distraction between the developers. We explore in more details the requirements of a tool to support DPP when we answer RQ3.

5.2.2.3. Teams characteristics. In relation to the team size, we did not found in the empirical studies a standard. Some of the studies conducted within universities give no details of how many students were in their experiment [5]. Zachari [34] has shown that 68 students were engaged in the study while in the experiment conducted by Baheti [1] 132 students were involved. Rosen et al. [23] did not mention how many professionals were involved in the pilot case study conducted in industry.

5.2.2.4. Metrics. The metrics adopted by DPP empirical studies to evaluate the variable and effects were very similar to those that were used for the PP studies [11,25]. Regarding productivity the most common metric used was LOC/h as identified in [1,5], and [34]. In the education setting, code quality and performance was measured by grades [1,13,34]. Metrics related to programmer perspective such as motivation and confidence were collected by surveys applied before and after the studies.

5.2.2.5. Impacts of the distribution. There are few aspects that DPP studies explored involving Global Software Development. Canfora et al. reported that due to the distance, the challenges in the practice are related with the lack of the common vision of project between the team. The authors suggest that a brainstorming with the distributed pairs could help to solve this [5]. Rosen et al. [23] also presented issues involving coordination in distributed pairs related with the difference between the goals of the developers in a DPP session. In a similar context Canfora [5] suggests a quick meeting before the session to align the goals of the pairs.

5.2.3. RQ3. Which are the main tools that can support the practice of DPP

In the literature, we identified a set of the tools to support DPP. Table 5 presents these tools and a short description.

Most of the specific DPP tools identified are an Eclipse plugin, confirming what Canfora reported about the importance of a tool be integrated in an existing software development tool [5]. And for some tools that are not an Eclipse plugin such as Collece, the authors reported the development of a plugin as future works [9].

VNC with NetMeeting [34] and Adobe Connect showed that DPP could also work with tools that were not designed specifically to support DPP. CollabVS is the only plugin identified that is not for Java platform. The majority of the specific tools are open source, but conference tools used for DPP have proprietary license.

Besides the screen sharing and a communication channel via text, voice or video, there are other important basic requirements that a DPP tool should have:

- **Shared Repository:** the pairs should access and manipulate files from the same repository [8,27].
- **Support the pairs regarding the specific roles:** one of the main problems in DPP is related to coordination between the pairs, for instance the fulfillment of the roles (driver and observer) [23]. The tool should support the coordination between both developers.
- **Support of roles between the pairs:** one of the main problems in DPP is related to coordination between the pairs, for instance the fulfillment of the roles (driver and observer) [23]. **The tool should support the coordination between the both programmers.**

- **Analysis of personality between the pairs:** Literature reported that an additional feature could be a recommendation system of pairs based on their personality [8].
- **Gesturing:** Hanks reported that a DPP tool might have a gesture feature to indicate parts of the code, accompanied by verbal cues such as “here” [14,15].

6. Discussion and conclusions

Both the quantitative and qualitative analyses present significant results regarding empirical evidence of DPP. This section aims to discuss such results.

6.1. Conclusions based on quantitative analysis

#1: There is a need for empirical studies of in industry DPP

Empirical studies involving DPP and real projects with industry professionals are very few. Therefore there is not much evidence about many variables related to DPP in industry. The study conducted by Rosen et al. is a first attempt that investigates DPP in a case study [23], presenting some challenges and benefits perceived in the adoption of the practice. However, there are also opportunities to evaluate other aspects of DPP, conducting ethnographies and observations that might present good results in extracting empirical evidence from industry settings. Another important observation is to analyze the relation between DPP and the software development approach that companies adopt. DPP is generally associated with the Extreme Programming agile method, and literature reports that this could involve several challenges, since several companies do not follow XP and adaptive approaches completely [10] or integrate DPP with a more prescriptive approach.

#2: There is a good opportunity for more empirical studies about DPP.

A recent survey conducted and published by Version One Inc. in 2013 [32] showed that 76% of the participants belong to a Distributed Software Development team. This number had increased compared to earlier replications of the same survey, where 65% of the participants were part of a distributed team [31]. This scenario confirms the trend of the increasing adoption of DDS in industry reported by Prikladnicki [21]. Although the importance of DDS in this context, DPP has been a recent area with few empirical studies reported from both the practice and the teaching perspectives. There is a need for studies with industry

Table 5
Tools for support DPP.

Study	Tool	Description	Specific DPP tool
	Jazz	A synchronous platform developed by IBM	-
[7]	COPPER	A synchronous source code editor that allows two distributed software developers to write a program using Pair Programming	X
[9]	Collece	A tool developed using Java technology, is a groupware system to support synchronous Distributed Pair Programming practices	X
[16]	Sangam	An Eclipse plug-in that has been specifically designed for DPP	X
[7]	Ripple	An Eclipse plug-in designed for educational settings that enable the developers to work remotely on the same Java program at the same time	X
[28]	Facetop	A collaborative system developed to facilitate Distributed Pair Programming while retaining a sense of physical presence for the users	X
[29]	XPairtise	An Eclipse plug-in that provides a platform for Distributed Pair Programming	X
[23,24]	Saros	An Eclipse plugin for Distributed Software Development	X
	VNC/	VNC is a graphical desktop sharing system that uses the Remote Frame Buffer protocol (RFB) to remotely control another	-
	NetMeeting	NetMeeting is a video conferencing tool	
[12]	Adobe Connect	A videoconferencing tool	
	CyberCollage	CyberCollage is a tool that enables groups of students to work in virtual Pair Programming	X
	Grewptool	A tool that supports distributed programming collaboration between the students	X
[27]	XEclip	An Eclipse plugin that supports DPP	X
	CollabVS	A Visual Studio plugin for DPP	X

professionals in order to consolidate the variables and the effects that can be generated with DPP adoption.

#3: There is a strong trend towards the use and research of the empirical effects of DPP in teaching programming.

In the literature, PP already proved to be an effective practice on teaching programming [25]. In the same way, some studies sought to analyze the effects of DPP on teaching. However, few studies explore DPP as a pedagogical tool and how DPP could be integrated with the trend of online courses for example. There is an opportunity for investigating the participation of a tutor (instructor) as Radermacher [22] did for PP. Different levels of course could be also investigate. While Zacharis [34], Hanks [13] and Edwards [12] executed empirical studies in introductory programming courses, one might be interested in investigating the impact of DPP adoption in more advanced levels of software programming courses. Another opportunity is to investigate the relation between DPP and the retention of students in the courses.

6.2. Conclusions based on qualitative analysis

#1: There is a need to investigate the effects of DPP with professionals.

Only one case study was reported in the literature related to the effects of DPP in industry [23]. As noted in the quantitative analysis, there is a need to investigate DPP in industry. Effects such as code quality, productivity and effort have been very little analyzed from the perspective of practice with professionals. Evidences would help and give support to the benefits that the practice can offer.

#2: The dispersion level of the teams was little contextualized in DPP.

Experiments involving DPP as reported by Canfora [5] and Hanks [13] did not provide proper context regarding the dispersion level of students when using DPP. These experiments just reported that students were geographically distant. The description of the context is important because it helps in identifying the characteristics of the environment where the practice was used. The main focus of the studies was on the effects of the practice of DPP, providing few details about the distribution of the teams. These results are consistent with a recent study reported by Smite [26], where the authors confirm that studies conducted with distributed teams do not have a proper description of the context, neither a standardization of terms to be used, making it difficult to compare empirical evidences.

#3: There is an opportunity to explore the effects of coordination, communication and cultural diversity in DPP.

Few studies found in the literature investigate the effect of challenges of distributed teams in the context of DPP. One of the main challenges is communication. Most experiments were done with people of the same country, so there was little interference in communication. It would be interesting to develop experiments and cases using teams from different countries with different primary languages and evaluate the effect of adopting DPP. Another challenge that could be explored is the time zone difference, and how the use of DPP is affected. Cultural diversity is also key issue in Distributed Software Development and often is stated as a challenge within distributed teams [6]. We do not identify any DPP study dealing with cultural aspects either in the practice and teaching perspectives or in the analysis of features in the specifics DPP tools that could support it. Some barriers such as political differences, motivation and work ethics should be considered in the allocation of the tasks or pairs.

#4: There is a need to explore more guidelines (lessons learned) for the implementation of DPP in the industry.

Canfora et al. [5] aimed to report lessons learned in a DPP experiment. Rosen et al. [23] also reported some problems and

the solutions experienced in case study in industry. Lessons learned are essential for the effective adoption of DPP and needed to be further explored.

#5 There is an opportunity to investigate the design, implementation and use of DPP tools.

The specific tools developed for DPP play an important role in order to support the collaboration and coordination within software development. As pointed out, the majority of DPP tools are plugins into consolidated IDEs (Integrated Development Environment) such as Eclipse [27] and Visual Studio, facilitating the shared editing and the integration with other features. However, there is very little empirical evaluation of such tools. XPairwise tool was evaluated based on the CVS (version management system) log [27]. Rosen et al. [23] empirically evaluated Saros tool using video recording in the DPP sessions and also with the log files generated by the tool. Since the use of a DPP tool is very important for the success of DPP adoption, investigating the design, implementation, use, and integration of DPP tool into the software development environment is an opportunity for future research.

#6: There is an opportunity to investigate DPP with other types of collaborative programming.

As presented and discussed in RQ2, most of the DPP studies compares the practice of DPP with PP and solo programming. However, few studies explore other types of programming such as Group Programming. In the literature, some studies reported that group programming (such as Coding Dojo Randori) presents good results in teaching aspects of programming, such as test driven development. Although some DPP tools enables more than two programmers, for instance Saros [23], there is no empirical evidence related to distributed group programming.

6.3. Limitations

As any other method, there are some limitations in conducted a Systematic Literature Review. The main limitations of this study are: the number and the sources (digital libraries) selected and the reliability of the paper classification method.

We did not look into every possible source. Eight digital libraries were selected based on past experiences of other researchers in conducting SLR [10,25]. We did not search for books, neither other sources of papers that could focus on studies from a non-scientific perspective. However, we believe that the results presented provide a good indication of the “state of the art” and the “state of the practice” of DPP.

The classification process based on the two perspectives (teaching or practice) could be subjective. To minimize this limitation, this process was always done in a peer review approach [18]. All papers were reviewed at least three times by the same researcher and discussed with a second researcher.

Acknowledgments

This research is partially funded by the agreement between ThoughtWorks and PUCRS. We also thank CNPq – Brazil (309000/2012-2).

Appendix A. Papers extracted from the SLR

This appendix provides a list of all the papers that provided input to this review, including those cited in the main paper and some additional references. The additional references were not included in the main paper because they are references that do not add any new information to those that are cited.

- [1] P. Baheti, E. Gehringer, D. Stotts, Exploring the efficacy of distributed pair programming, in: *Extreme Programming and Agile Methods—XP Agile Universe*, 2002, pp. 387–410.
- [2] M. Bandukda, Z. Nasir, Efficacy of distributed pair programming, in: *International Conference on Information and Emerging Technologies*, 2010, pp. 1–6.
- [3] E. Boyer, A. Dwight, A. Fondren, M. Vouk, J. Lester, A development environment for distributed synchronous collaborative programming, *SIGCSE Bull.* 40 (3) (2008) 158–162.
- [4] C. Bravo, R. Duque, J. Gallardo, A groupware system to support collaborative programming: design and experiences, *J. Syst. Softw.* 86 (7) (2013) 1759–1771.
- [5] G. Canfora, A. Cimitile, C. Visaggio, Lessons learned about distributed pair programming: what the knowledge needs to address? in: *Proceedings of the Twelfth IEEE International Conference of Enabling Technologies; Infrastructure for Collaborative Enterprises*, 2003, pp. 314–319.
- [6] G. Canfora, A. Cimitile, C. Visaggio, G. Di Lucca, How distribution affects the success of pair programming, *Int. J. Softw. Eng. Knowl. Eng.* (2006) 293–313.
- [7] J. Dajda, G. Dobrowolski, How to build support for distributed pair programming, in: *XP*, 2007.
- [8] P. Dewan, P. Agarwal, G. Shroff, R. Hegde, Distributed side-by-side programming, in: *ICSE Workshop on Cooperative and Human Aspects on Software Engineering*, 2009, CHASE '09, 17–17 May 2009, pp. 48–55.
- [9] W. Dou, K. Hong, X. Zhang, A framework of distributed pair programming system, in: *International Conference on Computational Intelligence and Software Engineering*, 2009, pp. 1–4.
- [10] W. Dou, W. He, Compatibility and requirements analysis of distributed pair programming, in: *Second International Workshop on Education Technology and Computer Science (ETCS)*, 6–7 March 2010, vol. 1, 2010, pp. 467–470.
- [11] W. Dou, W. He, A preliminary design of distributed pair programming system, in: *Second International Workshop on Education Technology and Computer Science (ETCS)*, 6–7 March 2010, vol. 1, 2010, pp. 256–259.
- [12] Dou, Kui Hong; Wei He, A conversation model of collaborative pair programming based on language/action theory, in: *14th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*, 14–16 April 2010, 2010, pp. 7–12.
- [13] B. Hanks, Distributed pair programming: an empirical evaluation, in: *XP*, 2004.
- [14] B. Hanks, Student performance in CS1 with distributed pair programming, *SIGCSE Bull.* (2005) 316–320.
- [15] B. Hanks, Empirical evaluation of distributed pair programming, *Int. J. Hum.–Comp. Stud.* 66 (7) (2008) 530–544.
- [16] B. Hanks, M. Brandt, Successful and unsuccessful problem solving approaches of novice programmers, *IGCSE Bull.* 41 (1) (2009) 24–28.
- [17] C. Ho, C. Raha, E. Gehringer, L. Williams, Sangam: a distributed pair programming plug-in for eclipse, in: *OOPSLA*, 2004, pp. 73–77.
- [18] A. Meneely, Laurie Williams, On preparing students for distributed software development with a synchronous, collaborative development platform, in: *Proceedings of the 40th ACM Technical Symposium on Computer Science Education (SIGCSE '09)*, 2009.
- [19] C. Murphy, Dan Phung, Gail Kaiser, A distance learning approach to teaching eXtreme programming, in: *Proceedings of the 13th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '08)*, ACM, New York, NY, USA, 2008, pp. 199–203.
- [20] K. Navoraphan, E. Gehringer, J. Culp, K. Gyllstrom, D. Stotts, Next-generation DPP with Sangam and Facetop, in: *Proceedings of the 2006 OOPSLA Workshop on Eclipse Technology eXchange*, 2006, pp. 6–10.
- [21] H. Natsu, J. Favela, A.L. Moran, D. Decouchant, A.M. Martinez-Enriquez, Distributed pair programming on the Web, in: *Proceedings of the Fourth Mexican International Conference on Computer Science*, 2003, ENC 2003, 8–12 September 2003, pp. 81–88.
- [22] A. Repenning, N. Ahmadi, N. Repenning, Andri Ioannidou, David Webb, Krista Marshall, Collective programming: making end-user programming (more) social, in: *Proceedings of the Third International Conference on End-User Development (IS-EUD'11)*, 2011.
- [23] E. Rosen, S. Salinger, C. Oezbek, Project kick-off with distributed pair programming, in: *Workshop of Psychology of Programming Interest Group*, Madrid, 2010, 15p.
- [24] T. Schümmer, S. Lukosch, Understanding tools and practices for distributed pair programming, *J. Univ. Comp. Sci.* 15 (16) (2010) 3101–3125.
- [25] A. Shaw, Extending the pair programming pedagogy to support remote collaborations in CS education, in: *Sixth International Conference on Information Technology: New Generations*, 2009, ITNG '09, 27–29 April 2009, pp. 1095–1099.
- [26] D. Stotts, L. Williams, N. Nagappan, P. Baheti, D. Jen, A. Jackson, Virtual teaming: experiments and experiences with distributed pair programming, in: *XP*, 2003.
- [27] D. Stotts, J. Smith, K. Gyllstrom, Support for distributed pair programming in the transparent video facetop, in: *XP*, 2004.
- [28] D. Tsompanoudi, M. Satratzemi, S. Xinogalos, Exploring the effects of collaboration scripts embedded in a distributed pair programming system, 2013.
- [29] N. Zacharis, Evaluating the effects of virtual pair programming on students' achievement and satisfaction, *Int. J. Emerg. Technol. Learn. (IJET)* 4 (3) (2009).
- [30] R.L. Edwards, J.K. Stewart, M. Ferati, Assessing the effectiveness of distributed pair programming for an online informatics curriculum, *ACM Inroads* 1 (1) (2010) 48–54.
- [31] S. Salinger, Christopher Oezbek, Karl Beecher, Julia Schenk, Saros: an eclipse plug-in for distributed party programming, in: *Proceedings of the 2010 ICSE Workshop on Cooperative and Human Aspects of Software Engineering (CHASE '10)*, 2010.
- [32] N. Zacharis, Measuring the effects of virtual pair programming in an introductory programming java course, *IEEE Trans. Edu.* 54 (2011) 168–170. Feb.
- [33] A.M. Zin, S. Idris, N.K. Subramaniam, Improving learning of programming through E-Learning by using asynchronous virtual pair programming, *Turk. Online J. Dist. Educ.* 7 (3) (2006) (article 13).

References

- [1] P. Baheti, E. Gehringer, D. Stotts, Exploring the efficacy of distributed pair programming, in: *Extreme Programming and Agile Methods—XP Agile Universe*, 2002, pp. 387–410.
- [2] K. Beck, C. Andres, *Extreme Programming Explained: Embrace Change*, second ed., Addison-Wesley Professional, 2004. p. 224.
- [3] P. Brereton, M. Turner, R. Kaur, Pair programming as a teaching tool: a student review of empirical studies, *Softw. Eng. Edu. Train.* (2009) 240–247.
- [4] G. Canfora, A. Cimitile, C. Visaggio, Lessons learned about distributed pair programming: what the knowledge needs to address? in: *Proceedings of the Twelfth IEEE International Conference of Enabling Technologies; Infrastructure for Collaborative Enterprises*, 2003, pp. 314–319.

- [5] G. Canfora, A. Cimitile, D. Visaggio, G. Di Lucca, How distribution affects the success of pair programming, *Int. J. Softw. Eng. Knowl. Eng.* (2006) 293–313.
- [6] S. Deshpande, I. Richardson, V. Casey, S. Beecham, Culture in global software development – a weakness or strength? in: 5th IEEE International Conference on Global Software Engineering (ICGSE), 2010, pp. 67–76, 23–26.
- [6a] P. Dewan, P. Agarwal, G. Shroff, R. Hegde, Distributed side-by-side programming, in: ICSE Workshop on Cooperative and Human Aspects on Software Engineering, 2009, CHASE '09, 17–17 May 2009, pp. 48–55.
- [7] W. Dou, K. Hong, X. Zhang, A framework of distributed pair programming system, in: International Conference on Computational Intelligence and Software Engineering, 2009, pp. 1–4.
- [8] W. Dou, W. He, Compatibility and requirements analysis of distributed pair programming, in: Second International Workshop on Education Technology and Computer Science (ETCS), 2010, vol. 1, pp. 467–470 (6–7 March 2010).
- [9] R. Duque, C. Bravo, Analyzing work productivity and program quality in collaborative programming, in: Proceedings of the 2008 The Third International Conference on Software Engineering Advances (ICSEA '08), IEEE Computer Society, Washington, DC, USA, pp. 270–276.
- [10] T. Dyba, T. Dingsøyr, Empirical studies of agile software development: a systematic review, *Inf. Softw. Technol. J.* 50 (10) (2008) 833–859 (Ago).
- [11] T. Dyba, E. Arisholm, D.I.K. Sjöberg, J.E. Hannay, F. Shull, The effectiveness of pair programming: a meta-analysis, *Inf. Softw. Technol.* 51 (7) (2009) 1110–1122.
- [12] R.L. Edwards, J.K. Stewart, M. Ferati, Assessing the effectiveness of distributed pair programming for an online informatics curriculum, *ACM Inroads* 1 (1) (2010) 48–54.
- [13] B. Hanks, Student performance in CS1 with distributed pair programming, in: SIGCSE Bulletin, 2005, pp. 316–320.
- [14] B. Hanks, Empirical evaluation of distributed pair programming, *Int. J. Hum. Comp. Stud.* 66 (7) (2007) 530–544.
- [15] B. Hanks, Problems encountered by novice pair programmers, *ACM J. Educ. Resour. Comput.* 7 (4) (2008).
- [16] J. Herbsleb, D. Moitra, Global software development, *IEEE Softw.*, California 16 (2) (2001) 16–20.
- [17] C. Ho, C. Raha, E. Gehringer, L. Williams, Sangam: a distributed pair programming plug-in for eclipse, in: OOPSLA, 2004, pp. 73–77.
- [18] B. Kitchenham, S. Charters, Guidelines for Performing Systematic Literature Reviews in Software Engineering, Technical Report, Keele University and Durham University, 2007, 65p.
- [19] C. McDowell, L. Werner, H. Bullock, J. Fernald, The effects of pair-programming on performance in an introductory programming course, in: SIGCSE Technical Symposium on Computer Science Education, 2002, pp. 38–42.
- [19a] K. Navoraphan, E. Gehringer, J. Culp, K. Gyllstrom, D. Stotts, Next-generation DPP with Sangam and Facetop, in: Proceedings of the 2006 OOPSLA Workshop on Eclipse Technology eXchange, 2006, pp. 6–10.
- [20] M. Paasivaara, S. Durasiewicz, C. Lassenius, Using scrum in distributed agile development: a multiple case study, in: ICGSE, 2009, pp. 195–204.
- [21] R. Prikladnicki, J.L.N. Audy, R. Evaristo, Global software development in practice: lessons learned, *Softw. Process Improve. Practice* 8 (2003) 267–281.
- [22] A. Radermacher, G. Walia, Investigating student-instructor interactions when using pair programming: an empirical study, in: 24th IEEE-CS Conference on Software Engineering Education and Training (CSEE&T), 2011, pp. 41–50, 22–24.
- [23] E. Rosen, S. Salinger, C. Oezbek, Project kick-off with distributed pair programming, in: Workshop of Psychology of Programming Interest Group, Madrid, 2010, 15p.
- [24] S. Salinger, Christopher Oezbek, Karl Beecher, Julia Schenk, Saros: an eclipse plug-in for distributed party programming, in: Proceedings of the 2010 ICSE Workshop on Cooperative and Human Aspects of Software Engineering (CHASE '10), 2010.
- [25] N. Salleh, E. Mendes, J. Grundy, Empirical studies of pair programming for CS/SE teaching in higher education: a systematic literature review, *IEEE Trans. Softw. Eng.* 37 (4) (2011) 509–525.
- [25a] D.T. Sato, H. Corbucci, M.V. Bravo, "Coding Dojo: An Environment for Learning and Sharing Agile Practices," *Agile, 2008. AGILE '08. Conference*, vol., no., pp. 459, 464, 4–8 Aug. 2008.
- [26] D. Smite, C. Wohlin, Z. Galvina, R. Prikladnicki, An empirically based terminology and taxonomy for global software engineering, *Emp. Softw. Eng.: Int. J.* (July) (2012).
- [27] T. Schümmer, S. Lukosch, Understanding tools and practices for distributed pair programming, *J. Univ. Comp. Sci.* 15 (16) (2010) 3101–3125.
- [28] D. Stotts, J. Smith, K. Gyllstrom, Support for distributed pair programming in the transparent video facetop, in: XP, 2004.
- [29] D. Tsompanoudi, M. Satratzemi, S. Xinogalos, Exploring the effects of collaboration scripts embedded in a distributed pair programming system, 2013.
- [30] J. Vanhanen, C. Lassenius, M.V. Mantyla, Issues and tactics when adopting pair programming: a longitudinal case study, in: *Software Engineering Advances*, 2007, pp. 25–31.
- [31] VERSIONONE INC, State of Agile Development Survey, VersionOne, Inc., Atlanta, 2011.
- [32] VERSIONONE INC, State of Agile Development Survey, VersionOne, Inc., Atlanta, 2013.
- [33] R. Wieringa, N. Maiden, C. Rolland, Requirements. Engineering paper classification and evaluation criteria: a proposal and a discussion, *J. Requir. Eng.* 11 (1) (2005) 102–107.
- [34] N. Zacharis, Measuring the effects of virtual pair programming in an introductory programming java course, *IEEE Trans. Edu.* 54 (2011) 168–170. Fev.
- [35] A.M. Zin, S. Idris, N.K. Subramaniam, Improving learning of programming through E-Learning by using asynchronous virtual pair programming, *Turk. Online J. Dist. Educ.* 7 (3) (2006) (article 13).